PRODUCT & SYSTEM DESIGN OVERVIEW

Adaptive Real-World Anomaly Intelligence System

ARAIS

Gap 1: Evaluation **Gap 2: Adaptability** **Gap 6: Open-World**

AI/ML Systems Architecture · April 2026

Document Purpose. This document defines the product form, interaction model, system behaviour, learning strategy, component architecture, and technology direction for ARAIS. It is a *design brief* — not an implementation specification. Every decision is justified against three constraints: **local compute only**, **no paid cloud services**, and the need to demonstrate production-grade intelligence within a single-developer or small-team build.

1. **Product Form & Architecture Decision**
2. **Core User Interaction Model**
3. **System Behaviour**
4. **Learning Strategy**
5. System Components
6. Technology Direction
7. Scope Boundaries
8. System Design Summary

1 1

1.1. What Are We Building?

ARAIS is a **local-first, desktop-grade intelligent anomaly detection workbench**. It runs entirely on the user's machine, accepts time-series and structured data, operates multiple anomaly detection models under a shared adaptive orchestration layer, and surfaces results through a rich dashboard — all without internet access or paid cloud backends.

It is positioned as a *developer and analyst workbench*: between a Jupyter notebook and a production monitoring tool, demonstrating serious system-level intelligence under constrained infrastructure.

1.2. Architecture Evaluation: Why Not Pure MERN?

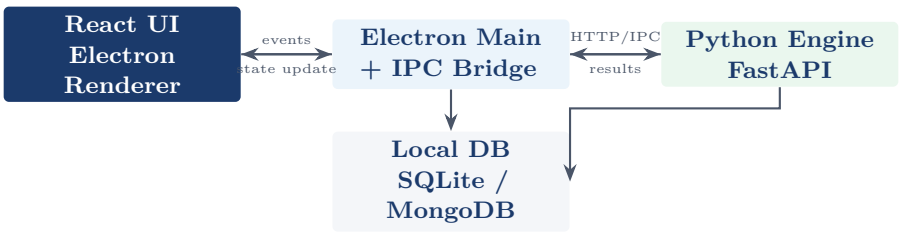
A standard MERN stack assumes a persistent server, a browser client, and a clear request/response boundary. Three structural mismatches arise for ARAIS:

1. **ML compute lives in Python.** Node.js has no credible native ML runtime. Every inference call must bridge to Python — which MERN has no clean mechanism for without a separate service layer.
2. **No server to host.** Without cloud, a "server" is a locally-running Express process the user must manually start — adding operational friction for a local tool.
3. **Browser sandboxing limits filesystem access.** File uploads, model checkpoints, and local database paths all require server mediation — unnecessary indirection for a local system.

React (the "R") remains valuable; the server model does not apply here.

1.3. Recommended Architecture

Electron provides a desktop shell embedding a Chromium renderer (React UI) and a Node.js main process (filesystem, IPC, database) — no separate server required. The user launches one executable; everything runs locally.
Python ML Engine runs as a managed child process spawned by Electron. Communication uses a lightweight local HTTP interface (FastAPI on localhost) or stdin/stdout IPC — zero infrastructure, zero latency, no external dependency.
Result: A single installable application with no server management, no browser restrictions, full filesystem access, full Python ML capability, and a production-grade React interface.



2 1

2.1. Inputs

Input Type	Description	Typical Source
Time-series files	Univariate/multivariate sequences: CPU metrics, sensor readings, log-derived streams	System telemetry, IoT logs
CSV / JSON upload	Tabular structured data with timestamp column and value columns	Analytics exports, DB dumps
Simulated stream	Internally generated real-time signal with configurable noise, drift, and anomaly injection	Built-in simulator

Three interaction modes:

1. **File upload via drag-and-drop.** Most common path. Users drag CSV/JSON onto the dashboard; the system parses and indexes locally.
2. **Local REST API (FastAPI).** Programmatic ingestion via POST requests — enabling integration with scripts, pipelines, or scheduled jobs on the same machine.
3. **Built-in stream simulator.** Generates time-series at configurable rates with Gaussian noise, step-change drift, and labelled anomaly events. Critical for demonstrating Gap 1 evaluation without requiring live external data.

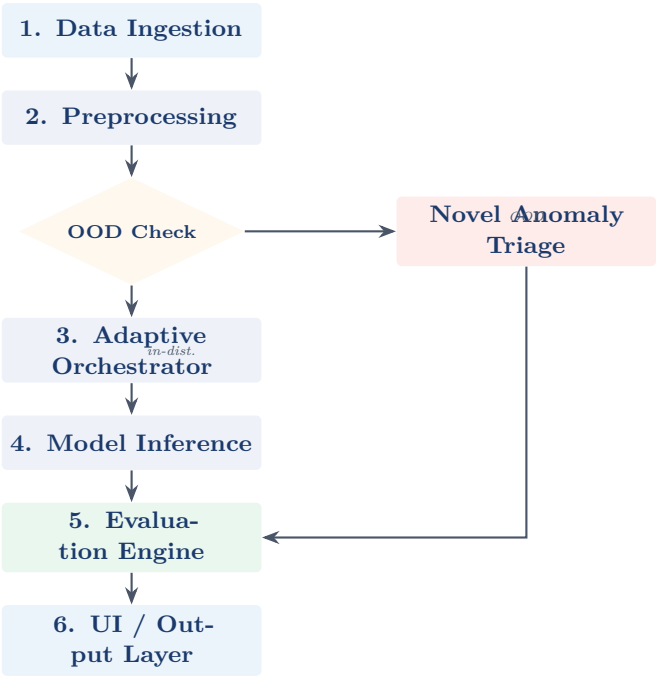
2.2. Outputs

Output	Meaning
Anomaly flag	Binary label `NORMAL`/`ANOMALY` per data point or window
Risk score	Continuous $[0, 1]$ severity score
Model confidence	Calibrated probability from the active detector
Epistemic uncertainty	Model-level uncertainty; high = model unsure about this input region
OOD proximity flag	Distance from training distribution; triggers novel-anomaly routing
Novel anomaly alert	Structured alert when input is detected as out-of-distribution
Evaluation report	Per-session performance: accuracy under noise/drift, latency histogram, switch log

ARAIS does not merely report anomalies — it communicates the *quality* of each prediction. A high-confidence normal differs fundamentally from a low-confidence normal. An OOD flag signals: "this is anomalous and unlike anything in training — do not trust my classification." These distinctions are first-class outputs, not footnotes.

3 1

3.1. Data Flow



Stage descriptions: (1) **Ingestion** — normalises formats and validates schema, emitting timestamped window buffers. (2) **Preprocessing** — applies windowing, normalisation, and optionally noise/drift injection; computes input statistics feeding the OOD check. (3) **OOD Check** — Mahalanobis/density-based distance test *before* inference; inputs beyond threshold route to Novel Anomaly Triage. (4) **Orchestration** — selects active model from the pool based on the runtime state vector (latency, CPU load, risk tier, live FP rate). (5) **Inference** — selected model returns anomaly prediction, confidence, and uncertainty. (6) **Evaluation** — every result is logged and profiled against current input conditions, closing the feedback loop. (7) **UI/Output** — live charts, alert panels, confidence gauges, and downloadable evaluation reports.

3.2. Adaptive Model Switching

The orchestrator maintains a *runtime state vector*: inference latency (moving average), CPU utilisation, session risk tier (`LOW`/`MEDIUM`/`HIGH`), and recent false-positive rate. Selection follows a priority rule:

- If latency budget exceeded **or** CPU above threshold → downgrade to lighter model.
- If risk tier is `HIGH` and resources available → upgrade to highest-accuracy model.
- Otherwise → consult the stored cost-accuracy frontier profile (populated offline).

Every switch event is logged with its triggering reason — providing full auditability.

3.3. Open-World / Unknown Anomaly Handling

Novel anomaly handling is three-step: **(a)** pre-inference distribution distance triggers OOD routing; **(b)** if inference still runs, high epistemic uncertainty flags the result as "uncertain anomaly"; **(c)** all OOD/high-uncertainty events are queued in the *novel event queue* — reviewable by the user and available as retraining data, forming the active-learning entry point.

4 1

Pre-trained models as the baseline · Local incremental fine-tuning as adaptation · Synthetic data as training substrate for stress-testing. No GPU farm. No cloud training API. No multi-gigabyte checkpoint downloads.

Pre-trained Baselines

The system ships with lightweight detectors trained on public benchmarks (NAB, KPI-Anomaly, SMD). They fit in CPU RAM, load in under 2 seconds, and produce usable results immediately — no user training required.

Local Incremental Training

When domain-specific data is uploaded, the system supports shallow incremental fine-tuning: adapting later model layers or normalisation statistics to the new distribution. No full retraining — goal is distribution adaptation, not architecture search.

Synthetic Evaluation Data

The stream simulator generates configurable anomaly injection, noise schedules, and concept drift — the primary vehicle for stress-testing Gap 1 conditions locally. Also serves as augmentation data when labelled real data is scarce.

Model Tiering

Tier	Model Type	Latency	When Used
Fast (Tier 1)	Z-score, EWMA, MAD	<5 ms	High load, low-risk sessions
Medium (Tier 2)	Isolation Forest, LOF, shallow LSTM	5–50 ms	Default operating mode
Deep (Tier 3)	Autoencoder, Transformer-lite, DeepSVDD	50–300 ms	Low load, high-risk sessions

5 1

Seven major components form the ARAIS operational architecture.

Accepts, validates, and normalises all incoming data regardless of source. Performs schema validation, timestamp parsing, missing-value flagging, and format unification into canonical timestamped window arrays. Owns the built-in stream simulator.

Transforms normalised input into model-ready feature windows. Applies sliding-window segmentation, z-score normalisation, optional lag/difference features, and configurable noise/drift injection in evaluation mode. Computes runtime input statistics that feed the OOD detection module.

Hosts and serves the tiered model pool. Each model exposes a uniform interface: receive a feature window → return anomaly score, confidence, and uncertainty estimate. Models are loaded lazily and cached. *No model is aware of the others*; all composition and switching is the orchestrator's sole responsibility.

The cognitive centre. Monitors the runtime state vector and selects the active model for each inference request. Maintains a cost-accuracy frontier profile per model, updated continuously from evaluation feedback. Logs every switch decision with its triggering reason — full auditability of adaptive behaviour.

Two-layer defence: (a) Pre-inference OOD screen using distribution distance metrics; (b) Post-inference uncertainty threshold check. All flagged events route to the novel event queue. Prevents silent misclassification of genuinely novel events.

Continuous, live model assessment — not a post-hoc batch report. Records every inference result alongside input conditions (noise estimate, drift rate, distribution distance). Builds and updates per-model performance profiles: precision, recall, FP rate, latency percentiles, all segmented by condition. Directly addresses Gap 1.

Four primary panels: (1) Live stream visualiser with anomaly overlays; (2) Inference result panel with confidence gauges and uncertainty indicators; (3) Model status panel — active model, recent switch events, cost-accuracy frontier; (4) Evaluation report viewer with condition-segmented performance profiles. React within Electron; all state local.

6 1

Technology	Role	Justification
Electron (v28+)	Desktop shell	Unified packaging; Node main process manages Python subprocess, filesystem, IPC
React (v18+)	UI framework	Real-time dashboards; Recharts/Victory for live stream visualisation
Python 3.11+	ML engine	scikit-learn, PyTorch, statsmodels ecosystem; runs as managed subprocess
FastAPI	Internal API	Lightweight local HTTP; typed endpoints, auto-docs, zero infrastructure
scikit-learn	Tier 1/2 models	Isolation Forest, LOF, OCSVM — CPU-native, no GPU required
PyTorch (CPU)	Tier 3 models	Autoencoder, lightweight Transformer; quantised models preferred
SQLite	Operational store	Inference log, evaluation records, session history; file-based, embedded
MongoDB (local)	Dataset storage	Flexible schema for ingested time-series and model metadata
NumPy / Pandas	Data processing	Standard time-series manipulation in the processing pipeline
River (optional)	Online learning	Incremental/online ML; natural fit for the adaptive fine-tuning path
ZeroMQ (optional)	Streaming IPC	If stdin/stdout proves insufficient for high-frequency streams

On MERN's place in this stack: The "M" (MongoDB) and "R" (React) from MERN are directly adopted. Express and Node-as-server are replaced by Electron's main process (IPC + OS integration) and Python/FastAPI (ML serving). This hybrid takes MERN's front-end tooling and discards the server model that does not suit local-first architecture.

7 1

Out of Scope	Reasoning
Distributed compute / cluster	Single-machine system; cluster infrastructure adds DevOps complexity with no research contribution.
Petabyte-scale data	Domain is sensor/log-scale streaming. Spark/Flink/Kafka is a distinct problem class.
Paid cloud services	Explicit constraint — no inference APIs, managed databases, or deployment pipelines.
Custom model architecture research	ARAIS orchestrates and evaluates models; it does not design new neural architectures.
Production SLA / enterprise hardening	No multi-tenancy, RBAC, or HA failover. Demonstration-grade platform, not production SaaS.
Automated full retraining	Incremental adaptation is in scope. Full retraining with hyperparameter search is explicitly deferred.

ARAIS is optimised for:

- **Realistic simulation** — noise injection, drift scheduling, OOD event generation, all locally
- **Demonstrable adaptive intelligence** — visible model switching, live cost-accuracy trade-offs, full auditability
- **Strong system design** — clean component separation, well-defined interfaces, closed-loop evaluation
- **Honest uncertainty communication** — every prediction carries confidence and OOD signals, not just a binary label

8 1

Dimension	Decision
Product form	Desktop application (Electron) — local-first, single executable
UI technology	React 18 within Electron renderer
ML runtime	Python 3.11 + FastAPI, managed subprocess
Communication	Electron IPC + local HTTP; ZeroMQ optional for high-frequency
Primary databases	SQLite (operational) + MongoDB local (dataset storage)
Model strategy	Pre-trained baselines + incremental fine-tuning; 3-tier pool
Evaluation posture	Continuous live profiling, not batch holdout
OOD handling	Pre-inference distribution check + post-inference uncertainty gate
Adaptivity mechanism	Runtime state vector → orchestrator → model switch with full audit log
Scope anchor	Single-machine, demonstration-grade, research-quality design